

Axilink Backend — API Documentation

Audience: an engineer building a **web version** of the Axilink client (the Flutter mobile app equivalent), who needs to talk to the same Spring Boot backend that the Android app and the desktop (Python) client already use.

1. What this backend actually is

There is **no REST/JSON HTTP API**. The whole backend is a single STOMP-over-WebSocket relay. It exposes one WebSocket endpoint, and every "endpoint" is really a STOMP **destination** (a pub/sub channel), not an HTTP route.

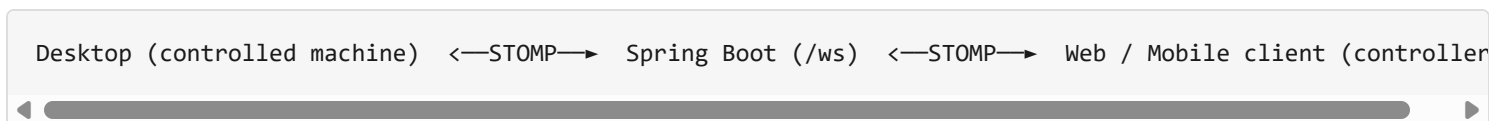
The server does almost nothing but re-broadcast: a client `SEND`s to

`/app/<thing>/<code>`, a `@MessageMapping` handler in `MotionController`

repackages it and calls `SimpMessagingTemplate.convertAndSend` to

`/topic/<thing>/<code>`, and every client subscribed to that topic receives

it. There is no database, no persistence, no auth beyond knowing the session code.



2. Connecting

- WebSocket endpoint: `ws://<desktop-host-ip>:8080/ws`
- Raw WebSocket (no SockJS fallback), CORS is wide open

(`setAllowedOrigins("*")` in `WebSocketConfig`).

- Protocol: STOMP 1.1/1.2 over that WebSocket.
- Heartbeat: server sends/expects a heartbeat every 10s (`10000`, `10000`).

If your STOMP client library supports heartbeats, enable them; otherwise the connection can be dropped as stale.

- Max frame size: 8 MB (text and binary). Payloads must stay under this.

The "session code"

There's no login. The desktop app, on startup, generates a short alphanumeric `session_code` and shows it as text + a QR code. Every STOMP destination is suffixed with that code, e.g. `/app/move/AB12CD` . Anyone who has the code (typed in manually, or by scanning the QR) can join that session — it's the only access control. A web client needs a screen where the user enters/scans this code before connecting.

3. Destinations reference

Two kinds of destinations:

- `/app/...` — where clients `SEND` commands *to* the backend. Handled

by `MotionController` (`Backend/Axilink Backend/src/main/java/.../controller/MotionController.java`).

The backend re-broadcasts the (possibly reshaped) payload to the matching

`/topic/...` .

- `/topic/...` — where clients `SUBSCRIBE` to receive broadcast data.

Direction	Destination	Who sends it	Payload
SEND	<code>/app/move/{code}</code>	Web/mobile controller	<code>MotionData</code>
SUBSCRIBE	<code>/topic/move/{code}</code>	Desktop client	relayed <code>MotionData</code> -shaped map
SEND	<code>/app/screen/{code}</code>	Desktop client	<code>ScreenFrameMessage</code>
SUBSCRIBE	<code>/topic/screen/{code}</code>	Web/mobile controller	relayed <code>ScreenFrameMessage</code> -shaped map
SEND	<code>/app/mode/{code}</code>	Web/mobile controller	<code>ModeMessage</code>
SUBSCRIBE	<code>/topic/mode/{code}</code>	Desktop client	<code>{ "mode": "..."</code> }
SEND	<code>/app/touch/{code}</code>	Web/mobile controller	<code>TouchEventMessage</code>

Direction	Destination	Who sends it	Payload
SUBSCRIBE	/topic/touch/{code}	Desktop client	relayed TouchEventMessage -shaped map
SEND (direct topic, see note)	/topic/ocr-screenshot/{code}	Desktop client	{ "image": "<base64 PNG>" }

Note on the OCR channel: the desktop client hand-crafts a raw STOMP `SEND` frame straight to `/topic/ocr-screenshot/{code}` instead of going through `/app/...`. Spring's simple broker relays any frame sent to a destination under its broker prefix (`/topic`) to that topic's subscribers, even though no controller method is mapped to it — there's no `@MessageMapping` handling this one. A web client only needs to `SUBSCRIBE` to `/topic/ocr-screenshot/{code}` ; it never sends to it.

4. Payload shapes

All bodies are JSON. Fields are optional unless stated otherwise — send only what's relevant to the action.

4.1 MotionData — /app/move/{code}

```
{
  dx?: number;           // mouse delta X, used when action is absent
  dy?: number;           // mouse delta Y, used when action is absent
  action?: string;        // see action values below
  scroll_dy?: number;      // only when action === "scroll"
  text?: string;          // only when action === "type"
}
```

Two shapes depending on intent:

- **Cursor movement** (gyroscope/trackpad delta): omit `action` , send

`{ dx, dy }` . The desktop multiplies these by a speed factor and moves the cursor relatively.

- **Discrete action**: set `action` to one of:

`left_click` , `right_click` , `double_click` , `scroll` (needs `scroll_dy`), `type` (needs `text`), `backspace` , `enter` .

(The Android app also emits `escape`, `copy`, `paste`, `undo`,

`select_all` via voice control, but the current desktop client doesn't

implement handlers for those yet — sending them is a no-op server-side

and a no-op on the desktop today.)

4.2 `ScreenFrameMessage` — `/app/screen/{code}` (desktop → controller)

The desktop streams JPEG screen frames as base64. Frames are large, so

they're sent either whole, split into **segments** (horizontal strips) or

chunks (byte-range pieces of a single segment/frame), depending on

size/network conditions. A web client's screen-mirror view needs to handle

all three shapes and reassemble by `frameId`.

```
{
  image?: string;           // whole frame, base64 JPEG
  imageSegment?: string;    // one horizontal strip, base64 JPEG
  segmentIndex?: number;
  totalSegments?: number;
  segmentY?: number;        // Y offset of this strip within the full frame
  segmentHeight?: number;   // height of this strip
  imageChunk?: string;      // one byte-range piece of a segment/frame, base64
  chunkIndex?: number;
  totalChunks?: number;
  frameId?: string;         // groups segments/chunks belonging to one frame
  aspectRatio?: number;     // width / height of the source screen
  timestamp?: number;       // epoch millis, for staleness/ordering
}
```

Rendering approach for a web client: keep a map keyed by `frameId`,

accumulate chunks in order, concatenate base64 per segment, and either

`` a full frame or draw segments onto

a `<canvas>` at `segmentY` once all segments for a `frameId` have arrived.

4.3 `ModeMessage` — `/app/mode/{code}`

```
{ mode: "mirror" | "control" }
```

Sent by the controller to tell the desktop/other peers which UI mode it's

in — `"mirror"` (screen-mirroring view) vs `"control"` (trackpad/gyro control view). Purely informational; the backend just relays it.

4.4 TouchEventMessage — `/app/touch/{code}`

Used in screen-mirror mode: the controller taps directly on the mirrored image, and coordinates are sent as **percentages of the mirrored frame** (0–100), which the desktop maps back to real screen pixels.

```
{
  xPercent: number; // 0-100, X position within the mirrored frame
  yPercent: number; // 0-100, Y position within the mirrored frame
  clickType: string; // "left_click" | "right_click" | "double_click" (default "left_click")
}
```

4.5 OCR screenshot — `/topic/ocr-screenshot/{code}` (subscribe only)

```
{ image: string } // base64 PNG, full resolution/lossless (larger than mirror frames)
```

Sent on-demand by the desktop when the controller requests an OCR capture (exact trigger lives in the Android app's OCR overlay flow, not in this backend).

5. Typical session flow

1. Desktop app starts, generates `session_code`, shows it + QR, opens its own WebSocket connection to `ws://<its-own-ip>:8080/ws` and subscribes to `/topic/move/{code}`, `/topic/mode/{code}`, `/topic/touch/{code}`.

1. Web client asks the user for the code (or scans the QR — QR payload is the code, or `code + ip` if the desktop is showing a remote/cloud URL).

1. Web client connects to the **same** `/ws` endpoint (same host:8080 the desktop is listening on, or a relay host if you add one) and:

- Subscribes to `/topic/screen/{code}` to render the mirror.

- Subscribes to `/topic/ocr-screenshot/{code}` if OCR is supported.
- Sends `/app/mode/{code}` once to announce its current mode.

1. As the user interacts (mouse-emulation gestures, taps on the mirrored

image, keyboard input), the web client sends to `/app/move/{code}` and

`/app/touch/{code}` .

1. Desktop receives these on `/topic/move/{code}` / `/topic/touch/{code}`

and drives `pyautogui` ; it captures the screen continuously and streams

frames via `/app/screen/{code}` , which the web client renders from

`/topic/screen/{code}` .

6. Web client implementation notes

- Use `@stomp/stompjs` (works directly over a native `WebSocket` , no

SockJS needed since the server allows raw WS with open CORS).

- Minimal connect:

```
import { Client } from '@stomp/stompjs';

const client = new Client({
  brokerURL: `ws://${host}:8080/ws`,
  heartbeatIncoming: 10000,
  heartbeatOutgoing: 10000,
  onConnect: () => {
    client.subscribe(`/topic/screen/${code}`, (msg) => renderFrame(JSON.parse(msg.body)));
    client.subscribe(`/topic/ocr-screenshot/${code}`, (msg) => renderOcr(JSON.parse(msg.body)));
    client.publish({ destination: `/app/mode/${code}`, body: JSON.stringify({ mode: 'mirror' }) });
  },
});
client.activate();

function sendTouch(xPercent: number, yPercent: number, clickType = 'left_click') {
  client.publish({
    destination: `/app/touch/${code}`,
    body: JSON.stringify({ xPercent, yPercent, clickType }),
  });
}
```

- Build screens to mirror what the Android app already provides:
- **Pairing screen:** enter/scan session code, connect.
- **Control screen:** trackpad-style area emitting `dx/dy` deltas (or

reuse gyroscope on mobile browsers via `DeviceOrientationEvent` if you

want parity with the phone app), buttons for `left_click` /

`right_click` / `double_click` / `scroll` , a text input wired to

`action: "type"` .

- **Mirror screen:** renders `/topic/screen/{code}` frames, forwards taps

as `xPercent` / `yPercent` via `/app/touch/{code}` .

- **On-screen keyboard:** same as mobile — each keypress sends

`{ action: "type", text: key }` , backspace/enter as their own actions.

- No reconnect/backoff logic exists server-side — implement it client-side

(the desktop Python client reconnects after 3s on close; mirror that).

- Nothing is authenticated beyond the session code — don't expose this

backend directly to the public internet without adding your own auth if

that's a requirement for the web version.

7. Running the backend

```
docker build -t axilink-backend .
docker run -p 8080:8080 axilink-backend
```

The container listens on `8080` (matches `server.port=8080` in

`application.properties`).